

文本匹配小案例二：词袋、TF-IDF 与余弦相似度

一、本节课要解决什么问题

在“小案例一”中，我们已经学会了几件很重要的事：

- 用 `jieba` 把一段中文文本切成词语；
- 去掉“的、了、和、以及”这类对判断主题帮助不大的停用词；
- 把“PySpark、pyspark、Spark”这类写法统一成标准技能词；
- 把一段原始简历或岗位描述，整理成更干净的词语列表。

但是，计算机还有一个关键问题没有解决：

计算机看不懂“熟悉 Python 和 Spark”这句话，它更擅长处理数字。

所以本节课要完成一次重要转换：

把文本变成数字，再用数学方法判断两段文本有多像。

在简历匹配项目中，这一步非常关键。因为系统最终要回答的问题是：

- 这份简历和这个岗位有多匹配？
- 哪些岗位更适合某个学生？
- 哪些简历更适合某个企业岗位？
- 匹配分数为什么高，为什么低？

本节课会学习三个核心概念：

- 1. 词袋模型：把文本变成“词语计数表”。
- 2. TF-IDF：判断哪些词真正重要。
- 3. 余弦相似度：计算两段文本的相似程度。

这三个概念看起来像算法，其实可以先把它们理解成生活中的三个动作：

算法概念	生活类比	它在做什么
词袋模型	清点购物袋里每种商品有几个	统计每个词出现了几次
TF-IDF	判断哪些商品最能代表这个购物袋	找出既常出现、又有区分度的词
余弦相似度	比较两个购物袋的组成比例像不像	判断两段文本方向是否相似

请注意：本节课不是为了背公式，而是为了理解一个流程：

文本清洗 → 词语统计 → 重要性加权 → 相似度计算 → 匹配排序

这就是后面“纯 Python/Pandas 版简历匹配项目”的核心地基。

二、课前准备

1. 本节课需要的 Python 包

如果你的环境中还没有安装 `scikit-learn` 和 `pandas`，可以在终端中执行：

```
pip install scikit-learn pandas -i https://mirrors.aliyun.com/pypi/simple/ --trusted-host mirrors.aliyun.com
```

如果你只做前面的手工计算案例，暂时不安装也可以。后面使用 `CountVectorizer`、`TfidfVectorizer`、`cosine_similarity` 时需要 `scikit-learn`。

2. 建议创建的代码文件夹

建议在项目中创建一个文件夹：

```
text_matching_demo_02
```

本讲义中的代码可以分别保存为：

```
01_bow_manual.py
02_bow_sklearn.py
03_tf_manual.py
04_idf_manual.py
05_tfidf_manual.py
06_tfidf_sklearn.py
07_cosine_manual.py
08_resume_job_similarity_case.py
09_multi_resume_job_topn.py
```

你也可以先把所有代码放在一个 `.py` 文件中逐段运行。正式练习时，建议分文件保存，更便于调试。

三、为什么文本必须变成数字

先看三段文本：

简历 A: 熟悉 Python、Spark、SQL, 做过数据分析项目
岗位 B: 要求 Python、Spark、Hadoop, 有数据处理经验
岗位 C: 要求 Java、Spring、MySQL, 负责后端开发

人一眼就能看出来:

- 简历 A 和岗位 B 更像;
- 简历 A 和岗位 C 不太像。

但计算机不能直接“感觉像不像”。它需要把文本变成类似下面这样的数字:

文本	Python	Spark	SQL	Hadoop	Java	Spring	MySQL
简历 A	1	1	1	0	0	0	0
岗位 B	1	1	0	1	0	0	0
岗位 C	0	0	0	0	1	1	1

这张表就像给每段文本做了一张“词语体检表”。有这个词就记 1, 没有就记 0。如果一个词出现多次, 也可以记录出现次数。

一旦文本变成数字, 计算机就能做数学计算。

四、词袋模型: 把文本装进一个“词语购物袋”

1. 什么是词袋模型

词袋模型的英文是 Bag of Words, 简称 BoW。

它的思想很朴素：

不管词语原来的顺序，只关心每个词出现了几次。

比如有两个购物袋：

购物袋 1：苹果、牛奶、面包、苹果

购物袋 2：牛奶、苹果、苹果、面包

这两个购物袋的顺序不一样，但里面的东西一样：

苹果：2 个

牛奶：1 个

面包：1 个

所以在词袋模型看来，它们是一样的。

把这个思想放到文本中：

文本 1：Python Spark SQL Python

文本 2：Spark Python Python SQL

词袋模型会认为这两段文本的词语组成完全一样：

Python：2

Spark：1

SQL：1

2. 词袋模型的优点和缺点

优点：

- 简单，容易理解；
- 适合初学者入门；
- 可以快速把文本转换成数字；
- 是 TF-IDF、相似度计算的基础。

缺点：

- 不关心词语顺序；
- 不理解句子含义；
- 不知道“熟悉 Python”和“不熟悉 Python”其实意思相反；
- 对同义词不敏感，比如“数据分析”和“数据挖掘”可能相关，但词面不同。

所以词袋模型不是万能的，但它是文本匹配入门最重要的一步。

五、案例 1：手工实现词袋模型

这个案例先不用任何机器学习库，只用 Python 基础语法实现词袋模型。

代码清单： `01_bow_manual.py`

```
# 01_bow_manual.py
# 目标：不用任何第三方库，手工实现一个最简单的词袋模型

# 假设这些词已经经过了分词、去停用词、技能词标准化
# 也就是说，这一步接在“小案例一”后面
# resume_tokens 表示一份学生简历中提取出来的关键词
# 这名学生的简历中提到了 python、spark、sql 和数据分析
# 其中 python 出现了两次，表示这个词在简历中被反复强调
resume_tokens = ["python", "spark", "sql", "数据分析", "python"]

# job_tokens 表示一个招聘岗位中提取出来的关键词
# 这个岗位要求 python、spark、hadoop 和数据处理能力
# 后面我们会比较 resume_tokens 和 job_tokens，看它们的关键词组成是否相似
job_tokens = ["python", "spark", "hadoop", "数据处理"]

# 第一步：把两段文本中出现过的所有词放到一个集合中
# set 可以自动去重
vocab_set = set(resume_tokens + job_tokens)

# 为了让输出顺序稳定，使用 sorted 排序
vocab = sorted(vocab_set)

print("词表：")
print(vocab)

def build_count_vector(tokens, vocab):
    """
    根据词表，把一段文本转换成词频向量。

    参数：
        tokens：一段文本分词后的结果，例如 ["python", "spark"]
    """
```

vocab: 统一词表, 例如 ["hadoop", "python", "spark"]

返回:

vector: 词频向量, 例如 [0, 1, 1]

"""

vector = []

for word in vocab:

tokens.count(word) 会统计 word 在 tokens 中出现了几次

count = tokens.count(word)

vector.append(count)

return vector

第二步: 分别把简历和岗位转换成数字向量

resume_vector = build_count_vector(resume_tokens, vocab)

job_vector = build_count_vector(job_tokens, vocab)

print("\n简历词频向量: ")

print(resume_vector)

print("\n岗位词频向量: ")

print(job_vector)

第三步: 为了更好理解, 把词和数字对应打印出来

print("\n词语与简历词频: ")

for word, count in zip(vocab, resume_vector):

print(word, "=>", count)

print("\n词语与岗位词频: ")


```
for word, count in zip(vocab, job_vector):  
    print(word, "=>", count)
```

运行后你应该理解什么

这段代码完成了三件事：

1. 建立统一词表；
2. 按照统一词表统计每段文本的词频；
3. 把文本转换成数字向量。

如果词表是：

```
['hadoop', 'python', 'spark', 'sql', '数据分析', '数据处理']
```

那么：

```
简历: ["python", "spark", "sql", "数据分析", "python"]
```

会变成：

```
[0, 2, 1, 1, 1, 0]
```

意思是：

词语	hadoop	python	spark	sql	数据分析	数据处理
简历词频	0	2	1	1	1	0

思考题 1

如果词表中有一个词“java”，但某份简历中没有出现它，这个位置应该填什么？

参考答案：

应该填 0。因为词表表示“所有可能被统计的词”，某段文本没有出现某个词，就说明它在该文本中的词频为 0。

思考题 2

为什么要使用统一词表，而不是每段文本各自建一个词表？

参考答案：

因为后面要比较两段文本。如果每段文本的词表不同，向量的每个位置含义就不同，无法直接比较。统一词表可以保证第 1 个位置、第 2 个位置在所有文本中表示同一个词。

六、案例 2：用 scikit-learn 实现词袋模型

手工写词袋模型可以帮助理解原理。实际项目中，我们通常使用成熟工具。

`scikit-learn` 中的 `CountVectorizer` 可以帮我们快速完成词袋统计。

代码清单：`02_bow_sklearn.py`

```
# 02_bow_sklearn.py
# 目标: 使用 scikit-learn 的 CountVectorizer 实现词袋模型

from sklearn.feature_extraction.text import CountVectorizer

# 注意:
# 这里为了简单演示, 假设文本已经分词, 并且词与词之间用空格隔开。
# 在中文处理中, CountVectorizer 默认不会帮我们做理想的中文分词。
# 所以更推荐先用 jieba 分词, 再用空格拼接成字符串。
documents = [
    "python spark sql 数据分析 python",
    "python spark hadoop 数据处理",
    "java spring mysql 后端开发",
]

# 创建词袋模型工具
# CountVectorizer 会负责两件事:
# 1. 从 documents 中学习词表
# 2. 按照词表统计每段文本中各个词出现的次数
vectorizer = CountVectorizer()

# fit_transform 会做两件事:
# 1. fit: 学习所有文本中的词表
# 2. transform: 把每段文本转换成词频向量
# 注意: 词表会保存在 vectorizer 对象内部
# count_matrix 保存的是每段文本对应的“数字矩阵”
count_matrix = vectorizer.fit_transform(documents)

# 获取词表
# get_feature_names_out() 会从 vectorizer 中取出刚才学习到的词表
# 词表中每个词的位置, 和词频矩阵中的列位置是一一对应的
feature_names = vectorizer.get_feature_names_out()
```

```
print("词表: ")
print(feature_names)

print("\n词频矩阵: ")
# count_matrix 默认是稀疏矩阵, 适合存储大量 0
# 初学阶段为了看清楚结果, 用 toarray() 转成普通二维数组
print(count_matrix.toarray())

# 逐行解释每段文本的词频
print("\n逐段文本词频: ")
# enumerate 可以在遍历每一行时, 同时拿到行号 doc_index
# vector 表示当前这段文本对应的词频向量
for doc_index, vector in enumerate(count_matrix.toarray()):
    # doc_index 从 0 开始, 所以输出时 +1, 更符合人的阅读习惯
    print(f"\n第 {doc_index + 1} 段文本: ")

    # zip(feature_names, vector) 会把“词语”和“词频数字”一一配对
    # 例如: ("python", 2)、("spark", 1)
    for word, count in zip(feature_names, vector):
        # 只打印出现过的词, 避免输出大量 0
        if count > 0:
            print(f"{word}: {count}")
```

这段代码中的关键词

代码	含义
<code>CountVectorizer()</code>	创建词袋模型工具
<code>fit_transform()</code>	学习词表并转换文本
<code>get_feature_names_out()</code>	查看词表
<code>toarray()</code>	把稀疏矩阵转换成普通数组，方便打印

为什么需要 `toarray()`

词袋模型在真实项目中可能产生非常大的矩阵。

比如有 1000 份简历，词表里有 5000 个词，那么矩阵大小就是：

```
1000 × 5000
```

但大多数文本只会包含其中一小部分词，所以矩阵中有大量 0。为了节省内存，`scikit-learn` 默认使用“稀疏矩阵”存储。

初学阶段，为了方便观察，可以使用：

```
# 把稀疏矩阵转换成普通二维数组，方便打印和观察
count_matrix.toarray()
```

把它转换成普通二维数组。

讨论题 1

为什么中文文本通常要先用 `jieba` 分词，再交给 `CountVectorizer` ？

参考答案：

因为英文天然用空格分隔单词，而中文句子中的词语没有空格边界。如果不先分词，工具可能无法正确识别“数据分析”“机器学习”“项目经验”等有意义的词组。先用 `jieba` 分词，再用空格拼接，可以让 `CountVectorizer` 按词统计。

七、TF：一个词在当前文本中有多常见

词袋模型只统计次数，但有时单纯次数会产生误导。

看两个文本：

文本 A: python spark sql

文本 B: python python python spark sql 数据分析 项目 经验

在文本 B 中，`python` 出现了 3 次，看起来非常重要。但文本 B 总词数也更多。

所以我们会关心：

某个词在当前文本中的出现比例是多少？

这就是 TF，英文是 Term Frequency，中文常译为“词频”。

一种简单计算方式是：

$$TF = \text{某个词在当前文本中出现的次数} / \text{当前文本的总词数}$$

比如：

文本: python python spark sql

总词数是 4:

$$\text{TF}(\text{python}) = 2 / 4 = 0.5$$
$$\text{TF}(\text{spark}) = 1 / 4 = 0.25$$
$$\text{TF}(\text{sql}) = 1 / 4 = 0.25$$

生活类比:

如果一个人的购物袋里有 10 件商品, 其中 5 件是水果, 那么水果比例是 50%。这说明这个购物袋很“水果化”。

同理, 如果一段简历文本里很多词都与 Python、Spark、数据分析有关, 那么这段简历就更偏向数据技术方向。

八、案例 3: 手工计算 TF

代码清单: 03_tf_manual.py

```
# 03_tf_manual.py
# 目标: 手工计算每个词的 TF 值

tokens = ["python", "python", "spark", "sql", "数据分析", "项目"]

def calculate_tf(tokens):
    """
    计算一段文本中每个词的 TF 值。

    TF = 某个词出现次数 / 当前文本总词数
    """
    # tf_dict 用来保存每个词的 TF
    # 计算前先暂时保存词出现次数
    tf_dict = {}

    # 当前文本总词数
    # 例如 tokens 中有 6 个词, total_count 就是 6
    total_count = len(tokens)

    for word in tokens:
        # 如果 word 第一次出现, 先设置为 0
        if word not in tf_dict:
            tf_dict[word] = 0

        # 每出现一次, 就加 1
        tf_dict[word] += 1

    # 把出现次数转换成比例
    for word in tf_dict:
        # 例如 python 出现 2 次, 总词数是 6
        # TF(python) = 2 / 6
```



```
tf_dict[word] = tf_dict[word] / total_count

return tf_dict


tf_result = calculate_tf(tokens)

print("TF 计算结果: ")
# items() 会同时取出词语和对应的 TF 值
for word, tf_value in tf_result.items():
    print(word, "=>", tf_value)
```

运行结果解读

如果输入是：

```
["python", "python", "spark", "sql", "数据分析", "项目"]
```

总词数是 6。

所以：

```
python = 2 / 6 = 0.3333
spark = 1 / 6 = 0.1667
sql = 1 / 6 = 0.1667
数据分析 = 1 / 6 = 0.1667
项目 = 1 / 6 = 0.1667
```

思考题 3

如果一个词出现次数很多，它的 TF 一定很高吗？

参考答案：

不一定。TF 不只看出现次数，还要看文本总长度。如果一个词出现 5 次，但整段文本有 500 个词，它的 TF 只有 0.01；如果一个词出现 2 次，但整段文本只有 10 个词，它的 TF 是 0.2，反而更高。

九、IDF：一个词能不能区分不同文本

TF 只看一个词在当前文本中是否常见，但它还不够。

假设班级通知中经常出现这些词：

同学、通知、要求、时间、安排

这些词出现频率很高，但它们不一定能说明通知的具体主题。

真正有区分度的词可能是：

奖学金、补考、体测、实训、答辩

因为这些词不是每篇通知都有，一旦出现，就更能说明这篇通知在讲什么。

这就是 IDF 的思想。

IDF 的英文是 Inverse Document Frequency，中文常译为“逆文档频率”。

它关心的是：

一个词在多少篇文档中出现过。

如果一个词在很多文档里都出现，说明它太常见，区分度低；

如果一个词只在少数文档里出现，说明它更特别，区分度高。

IDF 的常见公式

为了避免除以 0，并让计算更稳定，实际项目中常用带平滑的公式：

$$\text{IDF} = \log((\text{文档总数} + 1) / (\text{包含该词的文档数} + 1)) + 1$$

其中：

- 文档总数：一共有多少段文本；
- 包含该词的文档数：有多少段文本出现过这个词；
- `log`：对数函数，可以让数值变化更平稳；
- `+1`：平滑处理，避免极端情况。

为什么公式里要 `+1`

这个公式里其实有两处 `+1`：

$$\text{IDF} = \log((\text{文档总数} + 1) / (\text{包含该词的文档数} + 1)) + 1$$

第一处：

$$\frac{\text{文档总数} + 1}{\text{包含该词的文档数} + 1}$$

这叫**平滑处理**。它的主要作用是避免极端情况，让公式更稳定。

假设某个词暂时没有出现在任何文档中：

$$\text{包含该词的文档数} = 0$$

如果不用 $+1$ ，公式里就会出现：

$$\text{文档总数} / 0$$

除以 0 在数学和程序里都不允许。

加上 $+1$ 以后，就变成：

$$(\text{文档总数} + 1) / (0 + 1)$$

这样公式可以继续计算。

第二处：

$$\text{公式最后的} + 1$$

它的作用是让 IDF 的结果整体保持在一个更容易使用的范围里。

特别是当某个词出现在所有文档中时：

文档总数 = 包含该词的文档数

如果不看平滑，比例大约是：

文档总数 / 文档总数 = 1

而：

$\log(1) = 0$

这说明这个词完全没有区分度。

但在实际计算中，如果 IDF 直接变成 0，那么这个词的 TF-IDF 也会变成 0：

$TF-IDF = TF \times IDF = TF \times 0 = 0$

有些工具会选择保留一点基础权重，所以在最后再 +1，让常见词权重降低，但不一定完全消失。

可以这样理解：

分母里的 +1 是为了防止公式出问题；公式最后的 +1 是为了让权重更平稳、更适合后续计算。

为什么要取对数 $\log$

如果不取对数，少见词的 IDF 可能会变得过大。

举个例子，假设一共有 10000 篇文档：

词语	出现文档数	不取对数时的比例
python	5000	$10000 / 5000 = 2$
spark	100	$10000 / 100 = 100$
hadoop	10	$10000 / 10 = 1000$
very_rare_word	1	$10000 / 1 = 10000$

如果直接使用这些比例，极少出现的词会被放得特别大。

但少见不一定等于重要。它可能是：

- 一个真正重要的专业技能；
- 一个错别字；
- 一个姓名；
- 一个学校名；
- 一个偶然出现的无义词。

所以需要 `log` 把数值压缩一下。

例如：

```
log(10)    比 10 小很多
log(100)   比 100 小很多
log(10000) 也不会无限变大
```

`log` 的作用可以理解为：

承认少见词更有区分度，但不要让它因为太少见就“权重爆炸”。

生活类比：

如果全班 100 人里，90 人都有笔，这个物品很常见，区分度低；

如果只有 1 人带了一个很特殊的工具，它确实有区分度，但也不能只凭这个工具就完全判断这个人的专业方向。

所以 IDF 使用 `log`，就是为了让“少见词更重要”这件事变得温和一点。

`log` 的底数是多少

很多资料会把 IDF 公式统一写成：

```
IDF = log(...)
```

但严格来说，`log` 的底数可以有不同选择：

写法	含义	Python 写法
<code>ln(x)</code>	以 <code>e</code> 为底的自然对数	<code>math.log(x)</code>
<code>lg(x)</code>	以 <code>10</code> 为底的常用对数	<code>math.log10(x)</code>
<code>log2(x)</code>	以 <code>2</code> 为底的对数	<code>math.log2(x)</code>

本教程中的手工代码使用的是：

```
math.log(x)
```

所以这里的 `log` 实际上是自然对数 `ln`，底数是：

`e ≈ 2.71828`

`scikit-learn` 中的 `TfidfVectorizer` 默认使用的也是自然对数。

对文本匹配来说，底数不是最核心的问题。换不同底数，主要会改变 IDF 数值的缩放比例，但不会改变这个基本规律：

越常见的词，IDF 越低；
越少见的词，IDF 越高。

为了和 Python 代码以及 `scikit-learn` 保持一致，本课程默认把 `log` 理解为自然对数 `ln`。

不需要害怕这个公式。先记住一句话：

越常见的词，IDF 越低；越少见但有代表性的词，IDF 越高。

十、案例 4：手工计算 IDF

代码清单：`04_idf_manual.py`


```
# 04_idf_manual.py
# 目标: 手工计算每个词的 IDF 值

import math

# 假设有三段文本, 已经完成分词和清洗
# 每个内部列表代表一篇文档的分词结果
documents = [
    ["python", "spark", "sql", "数据分析"],
    ["python", "spark", "hadoop", "数据处理"],
    ["java", "spring", "mysql", "后端开发"],
]

def calculate_idf(documents):
    """
    计算所有文档中每个词的 IDF 值。

    使用平滑公式:
    
$$\text{IDF} = \log((\text{文档总数} + 1) / (\text{包含该词的文档数} + 1)) + 1$$

    """
    idf_dict = {}

    # total_docs 表示一共有多少篇文档
    # 这里是 3
    total_docs = len(documents)

    # 第一步: 建立总词表
    # vocab 中保存所有文档里出现过的不重复词
    vocab = set()
    for tokens in documents:
        # update 会把 tokens 中的所有词加入 vocab
```

```

# set 会自动去重
vocab.update(tokens)

# 第二步：逐个词计算它出现在多少篇文档中
for word in vocab:
    # doc_count 表示有多少篇文档包含当前 word
    doc_count = 0

    for tokens in documents:
        # 注意：这里统计的是“有多少篇文档出现过该词”
        # 一篇文档中出现 1 次和出现 10 次，都只算这篇文档包含该词
        if word in tokens:
            doc_count += 1

    # 第三步：套用 IDF 公式
    # 分母 doc_count + 1 是为了避免除以 0
    # 外面的 log 用来压缩数值，避免少见词权重过大
    # 最后的 +1 让 IDF 保持在更适合后续计算的范围内
    idf = math.log((total_docs + 1) / (doc_count + 1)) + 1

    # 把当前词的 IDF 保存到字典中
    idf_dict[word] = idf

return idf_dict

idf_result = calculate_idf(documents)

print("IDF 计算结果：")
# sorted 可以让输出顺序稳定，方便课堂对比
for word, idf_value in sorted(idf_result.items()):

```

```
# round(..., 4) 表示保留 4 位小数, 方便观察
print(word, "=>", round(idf_value, 4))
```

运行结果应该怎么看

你会发现：

- `python` 出现在第 1、2 段文本中，比较常见，IDF 较低；
- `spark` 也出现在第 1、2 段文本中，IDF 较低；
- `hadoop` 只出现在第 2 段文本中，IDF 较高；
- `java`、`spring`、`mysql` 只出现在第 3 段文本中，IDF 也较高。

讨论题 2

如果某个词在所有简历和岗位中都出现，例如“项目”“经验”，它对区分不同文本的帮助大吗？

参考答案：

帮助通常不大。因为所有文本都有这个词，它不能说明某一份简历或某一个岗位有什么特别之处。IDF 会降低这类常见词的权重。

讨论题 3

如果某个词只出现一次，它的 IDF 一定代表它很有用吗？

参考答案：

不一定。IDF 只能说明这个词少见，但少见不等于一定有用。它可能是重要技能词，也可能是错别字、无义词、姓名、学校名等。因此实际项目中还需要停用词、技能词表、人工规则等一起配合。

十一、TF-IDF：既看出现频率，也看区分能力

现在我们有了两个指标：

TF：一个词在当前文本中有多常见
IDF：一个词在全部文本中有多特别

把它们相乘，就得到 TF-IDF：

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

一句话理解：

TF-IDF 高的词，通常是在当前文本中比较常见、并且在其他文本中不那么泛滥的词。

生活类比：

如果你观察很多同学的书包，发现“笔”和“纸”几乎人人都有，它们不太能说明一个人的专业兴趣。

但如果某个同学书包里经常出现“Python 入门”“数据分析案例”“机器学习讲义”，这些物品就更能说明他的学习方向。

TF-IDF 做的事情类似：

- 常见但没区分度的词：权重降低；
- 当前文本中明显、且有区分度的词：权重提高。

十二、案例 5：手工计算 TF-IDF

代码清单: 05_tfidf_manual.py

```
# 05_tfidf_manual.py
# 目标: 手工计算 TF-IDF, 理解 TF 和 IDF 如何结合

import math

# documents 表示语料库, 也就是参与比较的所有文本集合
# 每个内部列表是一篇文档分词后的结果
documents = [
    ["python", "spark", "sql", "数据分析", "python"],
    ["python", "spark", "hadoop", "数据处理"],
    ["java", "spring", "mysql", "后端开发"],
]

def calculate_tf(tokens):
    """
    计算一篇文档中每个词的 TF。
    """
    # 先用字典统计每个词出现了几次
    tf_dict = {}

    # 当前文档总词数
    total_count = len(tokens)

    for word in tokens:
        # get(word, 0) 的意思是:
        # 如果 word 已经存在, 就取原来的次数
        # 如果 word 不存在, 就从 0 开始
        tf_dict[word] = tf_dict.get(word, 0) + 1

    for word in tf_dict:
        # 把“出现次数”转换成“出现比例”
```

```
tf_dict[word] = tf_dict[word] / total_count
```

```
return tf_dict
```

```
def calculate_idf(documents):
```

```
    """
```

```
    计算语料库中每个词的 IDF。
```

```
    语料库可以理解为：所有要参与比较的文本集合。
```

```
    """
```

```
    idf_dict = {}
```

```
    total_docs = len(documents)
```

```
    # 建立总词表
```

```
    # vocab 中保存所有文档里出现过的不重复词
```

```
    vocab = set()
```

```
    for tokens in documents:
```

```
        vocab.update(tokens)
```

```
    # 逐个词计算 IDF
```

```
    for word in vocab:
```

```
        # doc_count 表示当前词出现在多少篇文档中
```

```
        doc_count = 0
```

```
        for tokens in documents:
```

```
            if word in tokens:
```

```
                doc_count += 1
```

```
    # 使用平滑 IDF 公式
```

```
    # doc_count 越大, 说明词越常见, IDF 越低
```

```
    # doc_count 越小, 说明词越少见, IDF 越高
```

```
    idf_dict[word] = math.log((total_docs + 1) / (doc_count + 1)) + 1
```

```

return idf_dict

def calculate_tfidf(tokens, idf_dict):
    """
    计算一篇文档中每个词的 TF-IDF。
    """
    # 先计算当前文档内部的 TF
    tf_dict = calculate_tf(tokens)

    # 保存当前文档中每个词的 TF-IDF 分数
    tfidf_dict = {}

    for word, tf_value in tf_dict.items():
        # 从全局 IDF 字典中取出这个词的 IDF
        idf_value = idf_dict[word]

        # TF-IDF = TF * IDF
        # 当前文档中越常见、整体语料中越有区分度的词，分数越高
        tfidf_dict[word] = tf_value * idf_value

    return tfidf_dict

idf_dict = calculate_idf(documents)

print("第 1 篇文本的 TF-IDF: ")
# 这里只演示第 1 篇文本，也就是 documents[0]
tfidf_result = calculate_tfidf(documents[0], idf_dict)

# 按 TF-IDF 分数从高到低排序
# key=lambda item: item[1] 表示按字典项中的分数排序

```



```
for word, score in sorted(tfidf_result.items(), key=lambda item: item[1], reverse=True):  
    print(word, "=>", round(score, 4))
```

运行结果解读

在第一篇文本中：

```
["python", "spark", "sql", "数据分析", "python"]
```

`python` 出现了两次，TF 比较高；

但它在其他文本中也出现过，所以 IDF 不会特别高。

`sql`、`数据分析` 出现次数不多，但它们在其他文本中不常见，所以 IDF 更高。

这就是 TF-IDF 的平衡思想：

不能只看某个词出现几次，也要看它是不是有区分度。

思考题 4

为什么 TF-IDF 比单纯词频更适合做文本匹配？

参考答案：

因为单纯词频容易让常见词占据很高权重，而 TF-IDF 会降低全局常见词的影响，提高有区分度词语的权重。这样比较文本时，更容易关注真正能代表文本主题的词。

十三、案例 6：用 scikit-learn 计算 TF-IDF

实际项目中，我们通常不手工计算 TF-IDF，而是使用 `TfidfVectorizer`。

代码清单： `06_tfidf_sklearn.py`

```
# 06_tfidf_sklearn.py
# 目标: 使用 scikit-learn 的 TfidfVectorizer 计算 TF-IDF 矩阵

from sklearn.feature_extraction.text import TfidfVectorizer

# 这里的文本已经假设完成分词, 并用空格隔开
# TfidfVectorizer 会把空格分隔的词当作一个个 token
documents = [
    "python spark sql 数据分析 python",
    "python spark hadoop 数据处理",
    "java spring mysql 后端开发",
]

# 创建 TF-IDF 向量化工具
# 它会自动完成: 建词表、计算 TF、计算 IDF、生成 TF-IDF 矩阵
vectorizer = TfidfVectorizer()

# 学习词表, 并把文本转换成 TF-IDF 矩阵
# fit_transform 做两件事:
# 1. fit: 从所有文档中学习词表和 IDF
# 2. transform: 把每篇文档转换成 TF-IDF 向量
tfidf_matrix = vectorizer.fit_transform(documents)

# 获取词表
# 每个词的位置和 TF-IDF 矩阵中的列位置对应
feature_names = vectorizer.get_feature_names_out()

print("词表: ")
print(feature_names)

print("\nTF-IDF 矩阵: ")
# 初学阶段用 toarray() 把稀疏矩阵转成普通数组, 方便查看
```

```

print(tfidf_matrix.toarray())

print("\n逐段文本 TF-IDF 权重: ")
# 遍历每一篇文档的 TF-IDF 向量
for doc_index, vector in enumerate(tfidf_matrix.toarray()):
    print(f"\n第 {doc_index + 1} 段文本: ")

    # zip 把词语和对应权重配对
    word_scores = list(zip(feature_names, vector))

    # 按权重从高到低排序
    word_scores = sorted(word_scores, key=lambda item: item[1], reverse=True)

    for word, score in word_scores:
        # 只输出权重大于 0 的词
        # 权重为 0 表示该词没有出现在当前文档中
        if score > 0:
            print(f"{word}: {score:.4f}")

```

注意：这里的结果可能和手工计算不完全一样

你可能会发现，`TfidfVectorizer` 计算出的数字和我们手工案例中的数字不完全一致。

这是正常的。

原因包括：

- `scikit-learn` 默认会做向量归一化；
- 具体 TF 计算方式可能略有不同；
- IDF 平滑方式与参数设置有关。

但核心思想不变：

高频且有区分度的词，权重更高。

讨论题 4

如果一个岗位描述中出现了很多次“Python”，TF-IDF 一定会让它成为最高权重词吗？

参考答案：

不一定。如果“Python”在很多简历和岗位中都出现，它的 IDF 会下降。TF-IDF 会同时考虑当前文本中的出现频率和整个文本集合中的区分度。

十四、向量空间：把文本看成一个方向

现在我们已经能把文本变成数字向量。

比如词表是：

```
["python", "spark", "sql", "hadoop"]
```

简历 A：

```
[1, 1, 1, 0]
```

岗位 B：

[1, 1, 0, 1]

岗位 C:

[0, 0, 0, 0] 或 [0, 0, 0, 0, 其他方向]

向量可以理解成“方向”。

如果两个向量方向越接近，说明它们关注的词越相似；

如果两个向量方向差别很大，说明它们关注的内容不同。

生活类比：

两个人去超市购物：

- A 买了苹果、牛奶、面包；
- B 买了苹果、牛奶、酸奶；
- C 买了螺丝刀、电池、胶带。

A 和 B 的购物方向很像，都是食品类；

A 和 C 的购物方向差别大，一个像买早餐，一个像修东西。

文本向量也是这样。

十五、余弦相似度：比较两个向量的方向像不像

余弦相似度的英文是 Cosine Similarity。

它的核心思想是：

比较两个向量的夹角，夹角越小，越相似。

公式是：

$$\text{cosine_similarity}(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

公式拆开看：

符号	含义
$A \cdot B$	两个向量的点积
、	
、	
结果越接近 1	两个向量越相似
结果越接近 0	两个向量越不相似

初学时不用害怕公式，可以先记住：

余弦相似度更关注比例和方向，而不是文本绝对长度。

这对简历匹配很有用。

因为有的简历写得很长，有的简历写得很短。如果只看共同词数量，长简历可能天然占优势。余弦相似度可以在一定程度上缓解这个问题。

十六、案例 7：手工计算余弦相似度

代码清单：07_cosine_manual.py


```
# 07_cosine_manual.py
```

```
# 目标：不用第三方库，手工计算两个向量的余弦相似度
```

```
import math
```

```
def dot_product(vector_a, vector_b):
```

```
    """
```

```
    计算两个向量的点积。
```

```
    例如：
```

```
    A = [1, 2, 3]
```

```
    B = [4, 5, 6]
```

```
    点积 = 1*4 + 2*5 + 3*6
```

```
    """
```

```
    # result 用来累加每一维相乘后的结果
```

```
    result = 0
```

```
    # zip 会把两个向量相同位置的数字配成一对
```

```
    # 例如 [1,2,3] 和 [4,5,6] 会配成 (1,4)、(2,5)、(3,6)
```

```
    for a, b in zip(vector_a, vector_b):
```

```
        # 每一维相乘后加到总和里
```

```
        result += a * b
```

```
    return result
```

```
def vector_norm(vector):
```

```
    """
```

```
    计算向量长度。
```

```
    例如：
```

```
A = [3, 4]
长度 = sqrt(3*3 + 4*4) = 5
"""
# 先累加每一维的平方
square_sum = 0

for value in vector:
    square_sum += value * value

# 最后开平方，得到向量长度
return math.sqrt(square_sum)
```

```
def cosine_similarity(vector_a, vector_b):
    """
    计算两个向量的余弦相似度。
    """
    # 分子：两个向量的点积
    dot = dot_product(vector_a, vector_b)

    # 分母：两个向量长度的乘积
    norm_a = vector_norm(vector_a)
    norm_b = vector_norm(vector_b)

    # 如果某个向量长度为 0，说明它没有任何有效词
    # 这时无法计算方向，直接返回 0
    if norm_a == 0 or norm_b == 0:
        return 0

    # 余弦相似度 = 点积 / 两个向量长度的乘积
    return dot / (norm_a * norm_b)
```

```
resume_vector = [1, 1, 1, 0] # python, spark, sql, hadoop
job_vector_1 = [1, 1, 0, 1] # python, spark, hadoop
job_vector_2 = [0, 0, 0, 1] # hadoop

# 分别计算简历和两个岗位的相似度
score_1 = cosine_similarity(resume_vector, job_vector_1)
score_2 = cosine_similarity(resume_vector, job_vector_2)

# round(..., 4) 保留 4 位小数, 方便观察
print("简历 与 岗位 1 的相似度: ", round(score_1, 4))
print("简历 与 岗位 2 的相似度: ", round(score_2, 4))
```

运行结果解读

简历向量:

```
[1, 1, 1, 0]
```

岗位 1:

```
[1, 1, 0, 1]
```

它们共同拥有 `python` 和 `spark` , 所以相似度较高。

岗位 2:

```
[0, 0, 0, 1]
```

它只关注 `hadoop`，与简历没有共同方向，所以相似度较低。

思考题 5

为什么余弦相似度更适合比较文本方向，而不是简单比较共同词数量？

参考答案：

因为文本长短差异很大。简单共同词数量会让长文本更容易占优势，而余弦相似度会考虑向量长度，更关注词语组成比例和方向是否接近。

十七、案例 8：完整小案例，一份简历匹配多个岗位

现在把本节课的内容连起来：

1. 准备一份简历；
2. 准备多个岗位；
3. 使用 TF-IDF 把文本变成向量；
4. 使用余弦相似度计算匹配分；
5. 按分数排序，推荐最匹配的岗位。

代码清单： `08_resume_job_similarity_case.py`

```
# 08_resume_job_similarity_case.py
# 目标：使用 TF-IDF + 余弦相似度，完成一份简历匹配多个岗位的小案例

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 一份学生简历
# 为了聚焦本节课知识点，这里直接写成已经分词后的形式，用空格分隔
# 真实项目中，这个字符串应该来自“小案例一”的分词、停用词过滤和技能词标准化结果
resume_text = "python spark sql 数据分析 数据清洗 可视化 项目"

# 多个岗位描述
# 每个字典代表一个岗位
# job_name 是岗位名称，job_text 是岗位关键词文本
jobs = [
    {
        "job_name": "大数据开发实习生",
        "job_text": "python spark hadoop sql 数据处理 数据清洗",
    },
    {
        "job_name": "后端开发实习生",
        "job_text": "java spring mysql 接口开发 后端开发",
    },
    {
        "job_name": "数据分析实习生",
        "job_text": "python sql 数据分析 可视化 报表 分析",
    },
    {
        "job_name": "运维实习生",
        "job_text": "linux shell 服务器 网络 部署 运维",
    },
]
```

```
# 把简历和岗位文本放在同一个列表中
# 第 0 个是简历，后面都是岗位
# 这样做的原因是：简历和岗位必须使用同一套词表和同一套 IDF 权重
documents = [resume_text]

# 把每个岗位的 job_text 追加到 documents 中
for job in jobs:
    documents.append(job["job_text"])

# 使用 TF-IDF 把所有文本转换成向量
vectorizer = TfidfVectorizer()

# tfidf_matrix 的行数 = 1 份简历 + 多个岗位
# tfidf_matrix 的列数 = 所有文本中出现过的不重复词数量
tfidf_matrix = vectorizer.fit_transform(documents)

# 简历向量是第 0 行
# 因为 documents[0] 就是 resume_text
resume_vector = tfidf_matrix[0]

# 岗位向量是第 1 行到最后
# tfidf_matrix[1:] 表示从第 1 行取到最后一行
job_vectors = tfidf_matrix[1:]

# 计算简历和所有岗位的余弦相似度
# 结果是一个二维数组，例如 [[0.6, 0.1, 0.8, 0.0]]
# [0] 用来把二维结果取成一维分数列表
similarity_scores = cosine_similarity(resume_vector, job_vectors)[0]

# 把岗位名称和分数合并到一起
results = []
```

```
# zip(jobs, similarity_scores) 会把岗位和对应分数配对
# 例如：第一个岗位对应第一个分数
for job, score in zip(jobs, similarity_scores):
    results.append({
        "job_name": job["job_name"],
        "score": score,
    })

# 按匹配分从高到低排序
# reverse=True 表示降序，分数越高越靠前
results = sorted(results, key=lambda item: item["score"], reverse=True)

print("岗位匹配结果：")
# enumerate(..., start=1) 让排名从 1 开始
for rank, item in enumerate(results, start=1):
    print(f"{rank}. {item['job_name']}: {item['score']:.4f}")
```

运行结果可能类似

岗位匹配结果：

1. 数据分析实习生：0.5000
2. 大数据开发实习生：0.4300
3. 后端开发实习生：0.0000
4. 运维实习生：0.0000

具体数字可能因为版本和分词不同略有变化，但排序逻辑应该接近：

- 数据分析实习生：与简历中的 `python`、`sql`、`数据分析`、`可视化` 重合多；
- 大数据开发实习生：与 `python`、`spark`、`sql`、`数据清洗` 重合；

- 后端开发实习生：主要是 `java`、`spring`、`mysql`，方向不同；
- 运维实习生：主要是 `linux`、`shell`、`服务器`，方向也不同。

课堂讨论题 1

如果一个学生的简历很短，只写了“会 Python”，系统推荐结果可能准确吗？

参考答案：

可能不够准确。因为有效信息太少，系统只能根据“Python”这个词进行判断，容易把很多 Python 相关岗位都认为匹配。实际项目中需要引导学生补充项目经历、技能、工具、业务场景等信息。

课堂讨论题 2

为什么“数据分析实习生”和“大数据开发实习生”都可能排在前面？

参考答案：

因为简历中同时包含 `python`、`spark`、`sql`、`数据分析`、`数据清洗`、`可视化` 等词。其中一部分词偏数据分析，一部分词偏大数据处理，所以两个岗位都会有一定匹配度。

十八、案例 9：多个学生简历匹配多个岗位

真实项目中通常不是“一份简历匹配多个岗位”，而是：

多份简历 × 多个岗位

下面这个案例会输出每个学生最匹配的前 2 个岗位。

代码清单: 09_multi_resume_job_topn.py

```
# 09_multi_resume_job_topn.py
# 目标：多份简历匹配多个岗位，并输出每份简历最匹配的 Top N 岗位

import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# resumes 表示多份学生简历
# 每个字典是一份简历
# resume_text 是已经分词后的关键词文本
resumes = [
    {
        "resume_name": "张同学",
        "resume_text": "python spark sql 数据分析 数据清洗 可视化 项目",
    },
    {
        "resume_name": "李同学",
        "resume_text": "java spring mysql 后端开发 接口 项目",
    },
    {
        "resume_name": "王同学",
        "resume_text": "linux shell 服务器 部署 网络 运维",
    },
]

# jobs 表示多个岗位
# 每个岗位也有一个分词后的关键词文本 job_text
jobs = [
    {
        "job_name": "大数据开发实习生",
        "job_text": "python spark hadoop sql 数据处理 数据清洗",
    },
]
```

```
{
    "job_name": "后端开发实习生",
    "job_text": "java spring mysql 接口开发 后端开发",
},
{
    "job_name": "数据分析实习生",
    "job_text": "python sql 数据分析 可视化 报表 分析",
},
{
    "job_name": "运维实习生",
    "job_text": "linux shell 服务器 网络 部署 运维",
},
]
```

把所有简历和岗位文本放在一起建立统一词表

先单独取出所有简历文本

```
resume_texts = [item["resume_text"] for item in resumes]
```

再单独取出所有岗位文本

```
job_texts = [item["job_text"] for item in jobs]
```

合并后一起交给 TfidfVectorizer

这样简历和岗位会共享同一个词表和同一套 IDF 权重

```
documents = resume_texts + job_texts
```

使用 TF-IDF 转换文本

```
vectorizer = TfidfVectorizer()
```

```
tfidf_matrix = vectorizer.fit_transform(documents)
```

前 len(resumes) 行是简历向量

如果有 3 份简历, 那么第 0、1、2 行都是简历

```
resume_vectors = tfidf_matrix[:len(resumes)]
```

```
# 后 len(jobs) 行是岗位向量
# 从 len(resumes) 开始, 一直到最后, 都是岗位
job_vectors = tfidf_matrix[len(resumes):]

# 计算所有简历与所有岗位之间的相似度
# 矩阵形状: 简历数量 × 岗位数量
# 例如 3 份简历、4 个岗位, 结果就是 3 × 4 的矩阵
similarity_matrix = cosine_similarity(resume_vectors, job_vectors)

# 保存最终结果
# rows 中每一项会变成结果表的一行
rows = []

# 每份简历推荐前 top_n 个岗位
top_n = 2

# resume_index 是简历下标
# resume 是当前简历字典
for resume_index, resume in enumerate(resumes):
    # 取出当前简历和所有岗位的相似度分数
    # scores 的长度等于岗位数量
    scores = similarity_matrix[resume_index]

    # enumerate(scores) 会得到:
    # 岗位下标 + 对应匹配分
    job_score_pairs = list(enumerate(scores))

    # 按分数从高到低排序
    job_score_pairs = sorted(job_score_pairs, key=lambda item: item[1], reverse=True)

    # 只取前 top_n 个岗位
```

```
top_jobs = job_score_pairs[:top_n]

# 把当前简历的 Top N 推荐岗位写入 rows
for rank, (job_index, score) in enumerate(top_jobs, start=1):
    rows.append({
        "学生": resume["resume_name"],
        "推荐排名": rank,
        # job_index 用来从 jobs 列表中找到岗位名称
        "推荐岗位": jobs[job_index]["job_name"],
        # round(score, 4) 保留 4 位小数, 结果更整齐
        "匹配分": round(score, 4),
    })

# 转换成 DataFrame, 方便后续保存 CSV 或展示到 Streamlit
result_df = pd.DataFrame(rows)

# 打印推荐结果表
print(result_df)

# 如果需要保存结果, 可以取消下面这行注释
# utf-8-sig 可以减少中文 CSV 用 Excel 打开时的乱码问题
# result_df.to_csv("resume_job_topn_result.csv", index=False, encoding="utf-8-sig")
```

运行结果示例

	学生	推荐排名	推荐岗位	匹配分
0	张同学	1	数据分析实习生	0.4472
1	张同学	2	大数据开发实习生	0.3901
2	李同学	1	后端开发实习生	0.7905
3	李同学	2	大数据开发实习生	0.0000
4	王同学	1	运维实习生	0.9000
5	王同学	2	大数据开发实习生	0.0000

不同环境下分数可能略有变化，但每个学生的推荐方向应该比较合理。

讨论题 5

为什么有些第二推荐岗位的分可能是 0?

参考答案：

因为这份简历和该岗位没有共同的有效词，或者共同词在 TF-IDF 向量中没有形成有效相似方向。真实项目中如果分数为 0，选择不推荐，或者设置最低推荐阈值。

十九、如何把本节课接到真实简历匹配项目

本节课的案例使用的是已经分词好的文本：

```
"python spark sql 数据分析"
```

真实项目中，原始数据通常长这样：

"本人熟悉 Python、Spark 和 SQL，做过校园数据分析项目，能够完成数据清洗和可视化。"

所以完整流程应该是：

```
graph TD; A[原始简历 / 原始岗位] --> B[jieba 分词]; B --> C[去停用词]; C --> D[技能词标准化]; D --> E[用空格拼接成分词文本]; E --> F[TfidfVectorizer 转换成向量]; F --> G[cosine_similarity 计算相似度]; G --> H[排序推荐岗位或简历];
```

也就是说：

小案例一解决“文本怎么清洗”，小案例二解决“清洗后的文本怎么计算相似度”。

后面的完整项目，就是把这两部分拼起来，再加上数据读取、结果保存、页面展示。

二十、常见问题

1. TF-IDF 能真正理解文本意思吗

不能。

TF-IDF 主要基于词语统计，不理解深层语义。

例如：

```
我熟悉 Python  
我不熟悉 Python
```

这两句话都包含 `Python`，但意思相反。简单 TF-IDF 很难准确理解否定关系。

所以后续项目中，我们会结合规则分、多维评分、人工解释，让匹配结果更可靠。

2. 为什么有时候相似度很低，但人觉得应该匹配

可能原因包括：

- 两段文本使用了不同表达，比如“数据可视化”和“图表展示”；
- 技能词没有标准化；
- 分词结果不理想；
- 简历写得太少；
- 岗位描述太泛；
- TF-IDF 无法理解深层语义。

这也是为什么我们在“小案例一”中要做技能词标准化。

3. 是否可以只用关键词重合数量做匹配

可以做最简单版本，但效果通常不如 TF-IDF + 余弦相似度。

关键词重合数量的问题是：

- 不考虑文本长度；
- 不考虑词的重要程度；
- 常见词可能干扰结果；
- 很难处理多个岗位之间的细微差异。

4. 为什么简历和岗位要放在一起计算 TF-IDF

因为 TF-IDF 的 IDF 部分需要知道一个词在所有文本中有多常见。

如果简历和岗位分开计算，它们使用的词表和权重体系可能不同，不方便直接比较。

二十一、课堂练习

练习 1：修改词袋案例

请把 `01_bow_manual.py` 中的简历和岗位改成下面内容：

```
# 修改案例 1 中的简历关键词列表
resume_tokens = ["python", "flask", "mysql", "接口开发", "项目"]

# 修改案例 1 中的岗位关键词列表
job_tokens = ["python", "django", "mysql", "后端开发"]
```

要求：

1. 输出统一词表;
2. 输出简历词频向量;
3. 输出岗位词频向量;
4. 说明两个向量哪些位置相同。

参考答案:

两段文本共同包含 `python` 和 `mysql` , 所以这两个词对应位置的数值都大于 0。其他词如 `flask`、`django`、`接口开发`、`后端开发` 只出现在其中一段文本中出现。

练习 2: 解释 TF-IDF 权重

有三段文本:

文本 1: `python sql` 数据分析
文本 2: `python spark` 数据处理
文本 3: `python flask` 后端开发

问题:

1. `python` 的 IDF 大概会高还是低?
2. `spark` 的 IDF 大概会高还是低?
3. 哪个词更有区分度?

参考答案:

`python` 在三段文本中都出现, IDF 会比较低; `spark` 只在文本 2 中出现, IDF 会比较高。因此 `spark` 比 `python` 更能区分文本 2 的技术方向。

练习 3：解释余弦相似度

有三个向量：

```
简历 A: [1, 1, 0, 0]
岗位 B: [1, 1, 1, 0]
岗位 C: [0, 0, 1, 1]
```

问题：

1. 简历 A 更像岗位 B 还是岗位 C？
2. 为什么？

参考答案：

简历 A 更像岗位 B。因为简历 A 和岗位 B 在前两个维度上都有值，说明它们有共同关注的词；而岗位 C 的主要方向在后两个维度，与简历 A 没有重合。

练习 4：设计一个小型岗位推荐数据集

请自己设计：

- 2 份简历；
- 3 个岗位；
- 每段文本至少包含 5 个分词后的关键词；
- 使用 `TfidfVectorizer` 和 `cosine_similarity` 输出每份简历最匹配的岗位。

参考思路：

可以参考 `09_multi_resume_job_topn.py`，把 `resumes` 和 `jobs` 中的数据替换成自己设计的内容。注意每段文本中的词要用空格分隔。

二十二、本节课小结

本节课我们完成了文本匹配中非常关键的一步：

把清洗后的文本变成可以计算的数字，并用相似度方法判断匹配程度。

你需要掌握以下知识：

知识点	你应该能说清楚什么
词袋模型	如何把文本变成词频向量
TF	一个词在当前文本中出现比例
IDF	一个词在所有文本中的区分能力
TF-IDF	同时考虑词频和区分度
余弦相似度	比较两个文本向量方向是否接近
简历匹配流程	简历和岗位如何从文本变成排序结果

请记住本节课最重要的一句话：

文本匹配不是让计算机真正“读懂”文章，而是先把文本转换成一组有意义的数字，再用数学方法找出相似方向。

下一节课会继续往前推进：

学习 Word2Vec 词向量与语义相似度，让系统不仅看“词有没有重复”，也初步理解“词和词之间是否接近”。